
THEORY QUESTIONS

1. What is GUI Application? What does a GUI window include for the "Iris Flower Classification" task?

A GUI (Graphical User Interface) Application is a software application that allows users to interact with the system using graphical components such as buttons, labels, text boxes, and windows instead of command-line instructions.

For the "Iris Flower Classification" task, the GUI window should include:

1. Title of the application
2. Labels for input fields
3. Entry boxes for entering flower measurements:
    - Sepal Length
    - Sepal Width
    - Petal Length
    - Petal Width
4. Predict button to classify the flower
5. Message box to display prediction result
6. Error handling for invalid input
7. Fixed-size window for better interface control

The GUI is generally created using the Tkinter library in Python.

---

2. Explain the role of joblib and tkinter in creating a GUI Application.

Role of joblib

`joblib` is used to save and load trained machine learning models.

Functions:

- `joblib.dump()` → Saves trained model
- `joblib.load()` → Loads saved model

Purpose:

- Avoids retraining the model repeatedly
- Enables fast prediction in GUI applications

Example:

```
joblib.dump(model, "iris_model.pkl")
model = joblib.load("iris_model.pkl")
```

Role of tkinter

`tkinter` is Python's standard GUI library.

It is used to:

- Create windows
- Add labels and buttons
- Accept user input

- Display prediction results

GUI components:

- `Tk()`
- `Label`
- `Entry`
- `Button`
- `messagebox`

Thus, joblib handles model storage while tkinter handles user interaction.

---

3. How Error Handling is done in GUI Application?

Error handling in GUI applications is done using `try-except` blocks.

Purpose:

- Prevent program crash
- Handle invalid user input
- Display proper error messages

Common errors:

- Invalid numeric values
- Empty input fields
- Missing model file

Example:

```
try:
    value = float(entry.get())
except ValueError:
    messagebox.showerror("Error", "Invalid Input")
```

Benefits:

1. Improves reliability
2. Improves user experience
3. Prevents unexpected termination
4. Helps debugging

Thus, error handling makes GUI applications robust and user-friendly.

---

4. Explain User Interaction & Prediction for a GUI Application.

User interaction in a GUI application occurs through graphical components such as buttons, labels, and entry boxes.

Steps of interaction and prediction:

1. User enters feature values in Entry widgets.
2. User clicks the Predict button.
3. GUI collects input values.
4. Input values are converted into numeric form.

5. Saved model predicts the output using `model.predict()`.
6. Predicted flower category is displayed using messagebox.

Example:

```
prediction = model.predict([[sl, sw, pl, pw]])
```

The prediction result may be:

- Setosa
- Versicolor
- Virginica

Thus, GUI acts as an interface between the user and machine learning model.

---

5. Write a short note on the Program Flow of a GUI Application.

The program flow of a GUI application follows sequential steps from initialization to prediction.

Program Flow:

1. Import required libraries
2. Load trained model
3. Create GUI window
4. Add labels, entry boxes, and buttons
5. Accept user input
6. Validate input using error handling
7. Perform prediction
8. Display output
9. Run event loop using `mainloop()`

Flow:

```
Start → Load Model → Create GUI →
Take Input → Predict →
Display Result → End
```

Tkinter uses an event-driven model where actions occur when the user interacts with GUI components.

---

6. What is the main goal of Natural Language Processing (NLP)?

The main goal of Natural Language Processing (NLP) is to enable computers to understand, process, analyze, and generate human language.

NLP combines:

- Artificial Intelligence
- Machine Learning
- Linguistics

Objectives:

1. Understand text and speech
2. Extract meaning from language

3. Enable human-computer interaction

Applications:

- Chatbots
- Machine translation
- Speech recognition
- Sentiment analysis
- Text summarization

Thus, NLP helps machines communicate intelligently using natural language.

---

7. Which NLP task identifies names of people, organizations, and locations in text?

The NLP task that identifies names of people, organizations, and locations is called:

Named Entity Recognition (NER)

NER extracts important entities from text.

Examples:

- Person → Elon Musk
- Organization → Google
- Location → Mumbai

Example sentence:

```
"Ravi works at Microsoft in Bengaluru."
```

NER identifies:

- Ravi → Person
- Microsoft → Organization
- Bengaluru → Location

Applications:

- Information extraction
- Search engines
- Chatbots
- Document analysis

Thus, NER is an important NLP task used for identifying named entities.

---

8. What is the role of the discriminator in a Generative Adversarial Network (GAN)?

In a Generative Adversarial Network (GAN), the discriminator acts as a classifier that distinguishes between real and fake data.

GAN contains:

1. Generator
2. Discriminator

Role of Discriminator:

- Receives real and generated data
- Predicts whether data is real or fake
- Guides the generator to improve output quality

Working:

- If discriminator detects fake images correctly, generator learns to generate better images.

Objective:

```
Discriminator → Detect fake data
Generator → Fool discriminator
```

Thus, the discriminator improves the quality of generated data through adversarial learning.

---

9. Expand the term GPT in Generative AI.

GPT stands for:

Generative Pre-trained Transformer

Explanation:

- **Generative** → Generates text/content
- **Pre-trained** → Trained on large datasets before fine-tuning
- **Transformer** → Uses transformer neural network architecture

Features:

1. Understands context
2. Generates human-like text
3. Performs NLP tasks

Applications:

- Chatbots
- Content generation
- Translation
- Coding assistance

GPT models are widely used in modern Generative AI systems.

---

10. Which AI model is designed to generate images from text prompts?

The AI model designed to generate images from text prompts is called a:

Text-to-Image Generative Model

Examples:

- DALL·E
- Stable Diffusion

- Midjourney

Working:

1. User provides text prompt
2. Model interprets prompt
3. AI generates corresponding image

Example:

```
"A cat sitting on the moon"
```

The model creates an image matching the description.

Applications:

- Art generation
- Graphic design
- Advertising
- Game development

---

11. What does the term "hallucination" mean in Large Language Models (LLMs)?

Hallucination in Large Language Models refers to the generation of incorrect, misleading, or fabricated information that appears believable.

Characteristics:

1. Generates false facts confidently
2. Produces non-existing references
3. Creates inaccurate answers

Example:

- AI may generate fake book citations or incorrect historical facts.

Causes:

- Incomplete training data
- Statistical text generation
- Lack of real-world verification

Problems:

- Misinformation
- Reduced trust
- Incorrect decision making

Thus, hallucination is a major limitation of LLMs.

---

12. Name one major ethical concern related to LLMs.

One major ethical concern related to Large Language Models (LLMs) is:

Bias and Misinformation

LLMs may generate biased or harmful content because training data may contain societal biases.

Problems:

1. Gender bias
2. Racial bias
3. False information
4. Harmful stereotypes

Other ethical concerns:

- Privacy issues
- Deepfakes
- Copyright violations
- Misuse of AI-generated content

Therefore, ethical AI development and responsible usage are very important.

---

13. What is RLHF, and why is it used in ChatGPT training?

RLHF stands for:

Reinforcement Learning from Human Feedback

It is a technique used to improve AI model responses using human evaluations.

Working:

1. Humans rank model responses
2. AI learns preferred responses
3. Reinforcement learning updates the model

Purpose:

- Improve helpfulness
- Improve safety
- Reduce harmful outputs
- Align AI with human preferences

Benefits in ChatGPT:

1. More accurate answers
2. Better conversational quality
3. Safer responses
4. Improved user experience

Thus, RLHF helps train AI systems to behave more naturally and responsibly.

---

14. Differentiate between Zero-shot prompting and Few-shot prompting.

Aspect	Zero-shot Prompting	Few-shot Prompting
Definition	No examples are provided	Few examples are provided

Aspect	Zero-shot Prompting	Few-shot Prompting
Learning	Model answers directly	Model learns from examples
Guidance	Very little guidance	Better guidance
Accuracy	Lower for complex tasks	Higher accuracy
Example	"Translate this sentence"	Give 2–3 translation examples first

Zero-shot Example

```
Translate "Hello" to French.
```

Few-shot Example

```
Dog → Animal
Rose → Flower
Carrot → Vegetable
Apple → ?
```

Thus, few-shot prompting improves performance by providing examples.

---

15. What is the purpose of prompt engineering in Generative AI applications?

Prompt engineering is the process of designing effective prompts to obtain accurate and meaningful outputs from Generative AI models.

Purpose:

1. Improve response quality
2. Control AI behavior
3. Increase accuracy
4. Reduce ambiguity

Techniques:

- Clear instructions
- Role-based prompts
- Few-shot prompting
- Step-by-step prompting

Applications:

- Chatbots
- Code generation
- Content writing
- Image generation

Example:

```
"Explain Neural Networks in simple words with examples."
```

Thus, prompt engineering helps users communicate effectively with AI systems.

practical questions also, refering to code pdf , use same syntax , structure , code pattern as it in in code pdf

PRACTICAL QUESTIONS – Assignment 3

(Using same syntax style, structure, comments, and coding pattern from MLC 2 CODE PDF)

---

1. Write a program to save a trained machine learning model using `joblib.dump()`.

```
# SAVE TRAINED MODEL USING JOBLIB

# Required libraries
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
import joblib

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Create model
model = DecisionTreeClassifier()

# Train model
model.fit(X, y)

# Save model
joblib.dump(model, "iris_model.pkl")

print("Model saved successfully")
```

---

2. Write a Python program to load a saved model using `joblib.load()` and make predictions.

```
# LOAD SAVED MODEL AND MAKE PREDICTION

# Required libraries
import joblib
import numpy as np

# Load model
model = joblib.load("iris_model.pkl")

# Sample input
sample = np.array([[5.1, 3.5, 1.4, 0.2]])

# Prediction
prediction = model.predict(sample)

print("Predicted Class:", prediction)
```

---

3. Write a Tkinter program to create a GUI window with the title “Iris Flower Classification”.

```
# CREATE GUI WINDOW

# Tkinter required for GUI
import tkinter as tk

# Create window
root = tk.Tk()
```

```
# Window title
root.title("Iris Flower Classification")

# Window size
root.geometry("400x300")

# Run GUI
root.mainloop()
```

---

4. Write a Tkinter program to create labels and entry boxes for Sepal Length and Sepal Width.

```
# LABELS AND ENTRY BOXES

import tkinter as tk

root = tk.Tk()

root.title("Iris Flower Classification")

# Sepal Length
label1 = tk.Label(root, text="Sepal Length")
label1.pack()

entry1 = tk.Entry(root)
entry1.pack()

# Sepal Width
label2 = tk.Label(root, text="Sepal Width")
label2.pack()

entry2 = tk.Entry(root)
entry2.pack()

root.mainloop()
```

---

5. Write a Python program to create a button in Tkinter that displays a popup message when clicked.

```
# BUTTON WITH POPUP MESSAGE

import tkinter as tk
from tkinter import messagebox

# Function
def show_message():
    messagebox.showinfo("Message", "Button Clicked")

# Create window
root = tk.Tk()

root.title("Popup Example")

# Button
button = tk.Button(
    root,
    text="Click Me",
    command=show_message
```

```
)

button.pack()

root.mainloop()
```

---

6. Write a Python code snippet to handle invalid numeric input using try-except ValueError.

```
# ERROR HANDLING USING TRY-EXCEPT

try:
    value = float(input("Enter a number: "))
    print("Value:", value)

except ValueError:
    print("Invalid Numeric Input")
```

---

7. Write a program to check whether the file `iris_model.pkl` exists using the os module.

```
# CHECK FILE EXISTENCE

import os

# File check
if os.path.exists("iris_model.pkl"):
    print("Model file exists")

else:
    print("Model file not found")
```

---

8. Write a Python program to create four input fields using Tkinter Entry widgets.

```
# FOUR INPUT FIELDS USING ENTRY WIDGETS

import tkinter as tk

root = tk.Tk()

root.title("Iris Flower Classification")

# Input 1
entry1 = tk.Entry(root)
entry1.pack()

# Input 2
entry2 = tk.Entry(root)
entry2.pack()

# Input 3
entry3 = tk.Entry(root)
entry3.pack()

# Input 4
entry4 = tk.Entry(root)
entry4.pack()
```

```
root.mainloop()
```

---

9. Write code to predict the flower category using `model.predict()` for user-entered values.

```
# FLOWER PREDICTION USING MODEL

import joblib
import numpy as np

# Load model
model = joblib.load("iris_model.pkl")

# User values
sl = 5.1
sw = 3.5
pl = 1.4
pw = 0.2

# Prediction
prediction = model.predict([[sl, sw, pl, pw]])

print("Predicted Class:", prediction)
```

---

10. Write a Python program to display the predicted flower name using `messagebox.showinfo()`.

```
# DISPLAY PREDICTED FLOWER NAME

import tkinter as tk
from tkinter import messagebox

# Create window
root = tk.Tk()

root.withdraw()

# Predicted flower
flower = "Setosa"

# Display result
messagebox.showinfo(
    "Prediction",
    "Predicted Flower: " + flower
)
```

---

11. Write a program to create a fixed-size Tkinter window using `geometry()` and `resizable()`.

```
# FIXED SIZE WINDOW

import tkinter as tk

# Create window
root = tk.Tk()

root.title("Fixed Window")
```

```
# Window size
root.geometry("400x300")

# Disable resizing
root.resizable(False, False)

root.mainloop()
```

---

12. Write a Python program that loads the Iris dataset and prints the target names.

```
# LOAD IRIS DATASET

from sklearn.datasets import load_iris

# Load dataset
iris = load_iris()

# Print target names
print("Target Names:")

print(iris.target_names)
```